Tomasulo Multi-Issue Simulator

1 Architectural assumptions

- Issue width, writeback width, and commit width are the same number, from 1 to 4. It is the optional second command-line argument and defaults to 1. A value outside that range is clamped.
- Functional-unit counts are a separate cap from the issue width. Free units are counted once per cycle, so a unit that finishes later in that cycle is not reused. Two adders and three ready adds means two start.
- Reservation stations are their own cap. A full station or load/store queue refuses that instruction and everything behind it. The station stays busy until the operation is dispatched.
- The floating-point adder and the floating-point multiplier are pipelined: each unit accepts one new operation per cycle. The integer adder is not pipelined. With latency 1 it can still take a new add on the cycle after the previous one finishes. Address calculation and memory are not pipelined. Each load/store unit is one address slot and one memory slot.
- `Bne` and `Beq` are comparisons on the integer adder. They do not jump, do not stall fetch, and do not write a register. The PC advances when the branch issues, so a loop body runs once. The commit column stays empty.
- The stage order is issue, execute, memory, writeback, commit.
- A comment is recorded only when a rule actually blocks an instruction, and the blocked cycles follow the reason. A run of cycles prints as `[16-17]`. One cycle prints as `[8]`. The cycle the instruction issues, starts, or takes the bus is not included.
- `bus busy` names the missed writeback cycle. A result that finishes on cycle 27 can first write back on cycle 28, so a writeback on cycle 29 is `bus busy [28]`. The ordinary one-cycle latch is not a stall. If three results are waiting and one can write back on cycle x, the second is `bus busy [x]` and the third is `bus busy [x-x+1]`.
- The simulator keeps running while a memory unit is still writing. A store can leave the reorder buffer when it enters memory, and the write happens later.

2 Code changes

The starter already ran a single-issue Tomasulo pipeline. The changes below keep that pipeline and widen the points that were limited to one instruction.

The input parser reads the machine in one pass: reservation-station counts, functional-unit counts, execute and memory latencies, reorder-buffer size, and the initial registers and memory. Issue width is not read from the file. A file that contains only instructions still loads, using the starter's default machine. `Bne` and `Beq` are ordinary opcodes in that same table.

Each cycle the simulator issues, writes back, and commits up to the issue width, in order, and stops at the first instruction that cannot proceed. Dispatch starts one ready operation for each free unit.

- `station_stall` is the shared refusal when a reservation station or the load/store queue is full.
- `dispatch_alu` fills the integer adder, the floating-point adder, and the floating-point multiplier from the same rule.
- `ready_mem` is the memory eligibility list: a ready head store, or otherwise the loads the starter would accept.
- `age_entry` increments a functional-unit entry once per cycle and records the execute-start cycle. An entry already in the queue is aged by `fu_exe`. An entry that just entered is aged by `advance_new`, so older work is not stepped twice.
- `make_units` builds both the address units and the memory units.
- Stall comments carry the blocked cycles. Branches fall through instead of redirecting fetch.

3 How to run

From `code`:

```
python main.py test_cases/test_case.txt
python main.py test_cases/test_case.txt 4
```

The first command uses issue width 1. The second argument, when present, is the issue width for that run.

4 Results

Each subsection prints the input file, then the timing table for issue widths 1, 2, 3, and 4. Registers and memory are shown once. All four widths of a program leave that same state.

4.1 Loop body

Input file `test_case.txt`.

```
               # of rs         Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        3               4                               1
FP multiplier   2               15                              1
Load/store unit 5               1               5               1

ROB entries = 64
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

Ld F2 0(R1)
Mult.d F4 F2 F20
Ld F6 0(R2)
Add.d F6 F4 F6
Sd F6 0(R2)
Addi R1 R1 -4
Addi R2 R2 -4
Bne R1 R0 -12
Add.d F20 F2 F2
```

Issue width 1

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[9]	[10]	bus busy [8]
2	Mult.d F4 F2 F20	[2]	[10, 24]	[]	[25]	[26]	
3	Ld F6 0(R2)	[3]	[4, 4]	[8, 12]	[13]	[27]	no load/store unit [4-6]
4	Add.d F6 F4 F6	[4]	[26, 29]	[]	[30]	[31]	
5	Sd F6 0(R2)	[5]	[6, 6]	[32, 36]	[]	[32]	
6	Addi R1 R1 -4	[6]	[7, 7]	[]	[8]	[34]	
7	Addi R2 R2 -4	[7]	[8, 8]	[]	[10]	[35]	bus busy [9]
8	Bne R1 R0 -12	[8]	[9, 9]	[]	[]	[]	
9	Add.d F20 F2 F2	[9]	[10, 13]	[]	[14]	[37]	

Issue width 2

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
2	Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]	
3	Ld F6 0(R2)	[2]	[3, 3]	[8, 12]	[13]	[25]	no load/store unit [3-6]
4	Add.d F6 F4 F6	[2]	[25, 28]	[]	[29]	[30]	
5	Sd F6 0(R2)	[3]	[4, 4]	[31, 35]	[]	[31]	
6	Addi R1 R1 -4	[3]	[4, 4]	[]	[5]	[32]	
7	Addi R2 R2 -4	[4]	[5, 5]	[]	[6]	[33]	
8	Bne R1 R0 -12	[4]	[6, 6]	[]	[]	[]	
9	Add.d F20 F2 F2	[5]	[9, 12]	[]	[13]	[34]	

Issue width 3

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
2	Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]	
3	Ld F6 0(R2)	[1]	[2, 2]	[8, 12]	[13]	[25]	no load/store unit [1-6]
4	Add.d F6 F4 F6	[2]	[25, 28]	[]	[29]	[30]	
5	Sd F6 0(R2)	[2]	[3, 3]	[31, 35]	[]	[31]	no load/store unit [2]
6	Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[32]	
7	Addi R2 R2 -4	[3]	[4, 4]	[]	[5]	[32]	
8	Bne R1 R0 -12	[3]	[5, 5]	[]	[]	[]	
9	Add.d F20 F2 F2	[3]	[9, 12]	[]	[13]	[33]	

Issue width 4

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F2 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
2	Mult.d F4 F2 F20	[1]	[9, 23]	[]	[24]	[25]	
3	Ld F6 0(R2)	[1]	[2, 2]	[8, 12]	[13]	[25]	no load/store unit [1-6]
4	Add.d F6 F4 F6	[1]	[25, 28]	[]	[29]	[30]	
5	Sd F6 0(R2)	[2]	[3, 3]	[31, 35]	[]	[31]	no load/store unit [2]
6	Addi R1 R1 -4	[2]	[3, 3]	[]	[4]	[32]	
7	Addi R2 R2 -4	[2]	[3, 3]	[]	[4]	[32]	no integer adder [2]
8	Bne R1 R0 -12	[2]	[5, 5]	[]	[]	[]	
9	Add.d F20 F2 F2	[3]	[9, 12]	[]	[13]	[33]	

Registers and memory are the same at issue widths 1, 2, 3, and 4.

```
REGISTER VALUES

R0      R1      R2      R3      R4      R5      R6      R7
0       8       28      0       0       0       0       0
R8      R9      R10     R11     R12     R13     R14     R15
0       0       0       0       0       0       0       0
R16     R17     R18     R19     R20     R21     R22     R23
0       0       0       0       0       0       0       0
R24     R25     R26     R27     R28     R29     R30     R31
0       0       0       0       0       0       0       0
F0      F1      F2      F3      F4      F5      F6      F7
0       0       1.0     0       3.0     0       7.0     0
F8      F9      F10     F11     F12     F13     F14     F15
0       0       0       0       0       0       0       0
F16     F17     F18     F19     F20     F21     F22     F23
0       0       0       0       2.0     0       0       0
F24     F25     F26     F27     F28     F29     F30     F31
0       0       0       0       0       0       0       0

NON-ZERO MEMORY VALUES

Addresses   Values
4           3.0
8           2.0
12          1.0
24          6.0
28          5.0
32          7.0
```

4.2 Sample report 1

Input file `sample_report_01.txt`.

```
               # of rs         Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        4               3                               1
FP multiplier   4               5                               1
Load/store unit 4               1               6               1

ROB entries = 64
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

Issue width 1

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
2	Mult.d F2 F1 F0	[2]	[10, 14]	[]	[15]	[16]	
3	Add.d F3 F3 F2	[3]	[16, 18]	[]	[19]	[20]	
4	Mult.d F2 F1 F1	[4]	[11, 15]	[]	[16]	[21]	no fp multiplier [10]
5	Add.d F4 F2 F4	[5]	[17, 19]	[]	[20]	[22]	
6	Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[23]	
7	Ld F1 0(R1)	[7]	[9, 9]	[10, 15]	[17]	[24]	bus busy [16]
8	Mult.d F2 F1 F0	[8]	[18, 22]	[]	[23]	[25]	
9	Add.d F3 F3 F2	[9]	[24, 26]	[]	[27]	[28]	
10	Mult.d F2 F1 F1	[10]	[19, 23]	[]	[24]	[29]	no fp multiplier [18]
11	Add.d F4 F2 F4	[11]	[25, 27]	[]	[28]	[30]	
12	Addi R1 R1 8	[12]	[13, 13]	[]	[14]	[31]	
13	Ld F1 0(R1)	[13]	[15, 15]	[16, 21]	[22]	[32]	
14	Mult.d F2 F1 F0	[14]	[23, 27]	[]	[29]	[33]	bus busy [28]
15	Add.d F3 F3 F2	[17]	[30, 32]	[]	[33]	[34]	no reservation station [15-16]
16	Mult.d F2 F1 F1	[18]	[24, 28]	[]	[30]	[35]	no fp multiplier [23], bus busy [29]
17	Add.d F4 F2 F4	[19]	[31, 33]	[]	[34]	[36]	

Issue width 2

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
2	Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]	
3	Add.d F3 F3 F2	[2]	[16, 18]	[]	[19]	[20]	
4	Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[20]	no fp multiplier [10]
5	Add.d F4 F2 F4	[3]	[17, 19]	[]	[20]	[21]	
6	Addi R1 R1 8	[3]	[4, 4]	[]	[5]	[21]	
7	Ld F1 0(R1)	[4]	[6, 6]	[9, 14]	[15]	[22]	no load/store unit [6-7]
8	Mult.d F2 F1 F0	[4]	[16, 20]	[]	[21]	[22]	
9	Add.d F3 F3 F2	[5]	[22, 24]	[]	[25]	[26]	
10	Mult.d F2 F1 F1	[5]	[17, 21]	[]	[22]	[26]	no fp multiplier [16]
11	Add.d F4 F2 F4	[6]	[23, 25]	[]	[26]	[27]	
12	Addi R1 R1 8	[6]	[7, 7]	[]	[8]	[27]	
13	Ld F1 0(R1)	[7]	[9, 9]	[15, 20]	[21]	[28]	no load/store unit [9-13]
14	Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]	no reservation station [7-10]
15	Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]	no reservation station [11-16]
16	Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]	no fp multiplier [22]
17	Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]	

Issue width 3

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
2	Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]	
3	Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]	
4	Mult.d F2 F1 F1	[2]	[11, 15]	[]	[16]	[20]	no fp multiplier [10]
5	Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[21]	
6	Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[21]	
7	Ld F1 0(R1)	[3]	[5, 5]	[9, 14]	[15]	[21]	no load/store unit [5-7]
8	Mult.d F2 F1 F0	[3]	[16, 20]	[]	[21]	[22]	
9	Add.d F3 F3 F2	[3]	[22, 24]	[]	[25]	[26]	
10	Mult.d F2 F1 F1	[4]	[17, 21]	[]	[22]	[26]	no fp multiplier [16]
11	Add.d F4 F2 F4	[4]	[23, 25]	[]	[26]	[27]	
12	Addi R1 R1 8	[4]	[5, 5]	[]	[6]	[27]	
13	Ld F1 0(R1)	[5]	[7, 7]	[15, 20]	[21]	[27]	no load/store unit [7-13]
14	Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]	no reservation station [5-10]
15	Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]	no reservation station [11-16]
16	Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]	no fp multiplier [22]
17	Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]	no reservation station [17]

Issue width 4

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 8]	[9]	[10]	
2	Mult.d F2 F1 F0	[1]	[10, 14]	[]	[15]	[16]	
3	Add.d F3 F3 F2	[1]	[16, 18]	[]	[19]	[20]	
4	Mult.d F2 F1 F1	[1]	[11, 15]	[]	[16]	[20]	no fp multiplier [10]
5	Add.d F4 F2 F4	[2]	[17, 19]	[]	[20]	[21]	
6	Addi R1 R1 8	[2]	[3, 3]	[]	[4]	[21]	
7	Ld F1 0(R1)	[2]	[5, 5]	[9, 14]	[15]	[21]	no load/store unit [5-7]
8	Mult.d F2 F1 F0	[2]	[16, 20]	[]	[21]	[22]	
9	Add.d F3 F3 F2	[3]	[22, 24]	[]	[25]	[26]	
10	Mult.d F2 F1 F1	[3]	[17, 21]	[]	[22]	[26]	no fp multiplier [16]
11	Add.d F4 F2 F4	[3]	[23, 25]	[]	[26]	[27]	
12	Addi R1 R1 8	[3]	[5, 5]	[]	[6]	[27]	
13	Ld F1 0(R1)	[4]	[7, 7]	[15, 20]	[21]	[27]	no load/store unit [7-13]
14	Mult.d F2 F1 F0	[11]	[22, 26]	[]	[27]	[28]	no reservation station [4-10]
15	Add.d F3 F3 F2	[17]	[28, 30]	[]	[31]	[32]	no reservation station [11-16]
16	Mult.d F2 F1 F1	[17]	[23, 27]	[]	[28]	[32]	no fp multiplier [22]
17	Add.d F4 F2 F4	[18]	[29, 31]	[]	[32]	[33]	no reservation station [17]

Registers and memory are the same at issue widths 1, 2, 3, and 4.

```
REGISTER VALUES

R0      R1      R2      R3      R4      R5      R6      R7
0       28      32      0       0       0       0       0
R8      R9      R10     R11     R12     R13     R14     R15
0       0       0       0       0       0       0       0
R16     R17     R18     R19     R20     R21     R22     R23
0       0       0       0       0       0       0       0
R24     R25     R26     R27     R28     R29     R30     R31
0       0       0       0       0       0       0       0
F0      F1      F2      F3      F4      F5      F6      F7
0       5.0     25.0    0.0     26.0    0       0       0
F8      F9      F10     F11     F12     F13     F14     F15
0       0       0       0       0       0       0       0
F16     F17     F18     F19     F20     F21     F22     F23
0       0       0       0       3.0     0       0       0
F24     F25     F26     F27     F28     F29     F30     F31
0       0       0       0       0       0       0       0

NON-ZERO MEMORY VALUES

Addresses   Values
4           3.0
8           2.0
12          1.0
24          6.0
28          5.0
32          4.0
```

4.3 Sample report 2

Input file `sample_report_02.txt`.

```
              # of rs         Cycles in EX    Cycles in Mem   # of FUs
Integer adder   4               1                               1
FP adder        3               3                               2
FP multiplier   2               15                              2
Load/store unit 4               1               5               2

ROB entries = 128
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Addi R1 R1 4
Add.d F4 F2 F4
Addi R1 R1 8
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Ld F1 0(R1)
Mult.d F2 F1 F0
Add.d F3 F3 F2
Mult.d F2 F1 F1
Add.d F4 F2 F4
```

Issue width 1

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[9]	[10]	bus busy [8]
2	Mult.d F2 F1 F0	[2]	[10, 24]	[]	[25]	[26]	
3	Add.d F3 F3 F2	[3]	[26, 28]	[]	[29]	[30]	
4	Mult.d F2 F1 F1	[4]	[10, 24]	[]	[26]	[31]	bus busy [25]
5	Ld F1 0(R1)	[5]	[6, 6]	[7, 11]	[12]	[32]	
6	Addi R1 R1 4	[6]	[7, 7]	[]	[8]	[33]	
7	Add.d F4 F2 F4	[7]	[27, 29]	[]	[30]	[34]	
8	Addi R1 R1 8	[8]	[9, 9]	[]	[10]	[35]	
9	Ld F1 0(R1)	[9]	[11, 11]	[12, 16]	[17]	[36]	
10	Mult.d F2 F1 F0	[11]	[18, 32]	[]	[33]	[37]	no reservation station [10]
11	Add.d F3 F3 F2	[12]	[34, 36]	[]	[37]	[38]	
12	Mult.d F2 F1 F1	[13]	[18, 32]	[]	[34]	[39]	bus busy [33]
13	Ld F1 0(R1)	[14]	[15, 15]	[16, 20]	[21]	[40]	
14	Mult.d F2 F1 F0	[19]	[22, 36]	[]	[38]	[41]	no reservation station [15-18], bus busy [37]
15	Add.d F3 F3 F2	[27]	[39, 41]	[]	[42]	[43]	no reservation station [20-26]
16	Mult.d F2 F1 F1	[28]	[29, 43]	[]	[44]	[45]	
17	Add.d F4 F2 F4	[29]	[45, 47]	[]	[48]	[49]	

Issue width 2

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
2	Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]	
3	Add.d F3 F3 F2	[2]	[25, 27]	[]	[28]	[29]	
4	Mult.d F2 F1 F1	[2]	[9, 23]	[]	[24]	[29]	
5	Ld F1 0(R1)	[3]	[4, 4]	[5, 9]	[10]	[30]	
6	Addi R1 R1 4	[3]	[4, 4]	[]	[5]	[30]	
7	Add.d F4 F2 F4	[4]	[25, 27]	[]	[28]	[31]	
8	Addi R1 R1 8	[4]	[6, 6]	[]	[7]	[31]	
9	Ld F1 0(R1)	[5]	[8, 8]	[9, 13]	[14]	[32]	
10	Mult.d F2 F1 F0	[10]	[15, 29]	[]	[30]	[32]	no reservation station [5-9]
11	Add.d F3 F3 F2	[10]	[31, 33]	[]	[34]	[35]	
12	Mult.d F2 F1 F1	[11]	[15, 29]	[]	[30]	[35]	
13	Ld F1 0(R1)	[11]	[12, 12]	[13, 17]	[18]	[36]	
14	Mult.d F2 F1 F0	[16]	[19, 33]	[]	[34]	[36]	no reservation station [12-15]
15	Add.d F3 F3 F2	[26]	[35, 37]	[]	[38]	[39]	no reservation station [16-25]
16	Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[43]	
17	Add.d F4 F2 F4	[27]	[43, 45]	[]	[46]	[47]	

Issue width 3

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
2	Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]	
3	Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]	
4	Mult.d F2 F1 F1	[2]	[9, 23]	[]	[24]	[29]	
5	Ld F1 0(R1)	[2]	[3, 3]	[4, 8]	[9]	[29]	
6	Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[30]	
7	Add.d F4 F2 F4	[3]	[25, 27]	[]	[28]	[30]	
8	Addi R1 R1 8	[3]	[5, 5]	[]	[6]	[30]	
9	Ld F1 0(R1)	[3]	[7, 7]	[8, 12]	[13]	[31]	
10	Mult.d F2 F1 F0	[10]	[14, 28]	[]	[29]	[31]	no reservation station [4-9]
11	Add.d F3 F3 F2	[10]	[30, 32]	[]	[33]	[34]	
12	Mult.d F2 F1 F1	[10]	[14, 28]	[]	[29]	[34]	
13	Ld F1 0(R1)	[11]	[12, 12]	[13, 17]	[18]	[34]	
14	Mult.d F2 F1 F0	[15]	[19, 33]	[]	[34]	[35]	no reservation station [11-14]
15	Add.d F3 F3 F2	[26]	[35, 37]	[]	[38]	[39]	no reservation station [15-25]
16	Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[43]	
17	Add.d F4 F2 F4	[26]	[43, 45]	[]	[46]	[47]	

Issue width 4

Insn	Instruction	Issue	Exe	Mem	WB	Commit	Comments
1	Ld F1 0(R1)	[1]	[2, 2]	[3, 7]	[8]	[9]	
2	Mult.d F2 F1 F0	[1]	[9, 23]	[]	[24]	[25]	
3	Add.d F3 F3 F2	[1]	[25, 27]	[]	[28]	[29]	
4	Mult.d F2 F1 F1	[1]	[9, 23]	[]	[24]	[29]	
5	Ld F1 0(R1)	[2]	[3, 3]	[4, 8]	[9]	[29]	
6	Addi R1 R1 4	[2]	[3, 3]	[]	[4]	[29]	
7	Add.d F4 F2 F4	[2]	[25, 27]	[]	[28]	[30]	
8	Addi R1 R1 8	[2]	[5, 5]	[]	[6]	[30]	
9	Ld F1 0(R1)	[3]	[7, 7]	[8, 12]	[13]	[30]	
10	Mult.d F2 F1 F0	[10]	[14, 28]	[]	[29]	[30]	no reservation station [3-9]
11	Add.d F3 F3 F2	[10]	[30, 32]	[]	[33]	[34]	
12	Mult.d F2 F1 F1	[10]	[14, 28]	[]	[29]	[34]	
13	Ld F1 0(R1)	[10]	[11, 11]	[12, 16]	[17]	[34]	
14	Mult.d F2 F1 F0	[15]	[18, 32]	[]	[33]	[34]	no reservation station [11-14]
15	Add.d F3 F3 F2	[26]	[34, 36]	[]	[37]	[38]	no reservation station [15-25]
16	Mult.d F2 F1 F1	[26]	[27, 41]	[]	[42]	[43]	
17	Add.d F4 F2 F4	[26]	[43, 45]	[]	[46]	[47]	

Registers and memory are the same at issue widths 1, 2, 3, and 4.

```
REGISTER VALUES

R0      R1      R2      R3      R4      R5      R6      R7
0       24      32      0       0       0       0       0
R8      R9      R10     R11     R12     R13     R14     R15
0       0       0       0       0       0       0       0
R16     R17     R18     R19     R20     R21     R22     R23
0       0       0       0       0       0       0       0
R24     R25     R26     R27     R28     R29     R30     R31
0       0       0       0       0       0       0       0
F0      F1      F2      F3      F4      F5      F6      F7
0       6.0     36.0    0.0     37.0    0       0       0
F8      F9      F10     F11     F12     F13     F14     F15
0       0       0       0       0       0       0       0
F16     F17     F18     F19     F20     F21     F22     F23
0       0       0       0       3.0     0       0       0
F24     F25     F26     F27     F28     F29     F30     F31
0       0       0       0       0       0       0       0

NON-ZERO MEMORY VALUES

Addresses   Values
4           3.0
8           2.0
12          1.0
24          6.0
28          5.0
32          4.0
```

5 Discrepancies with existing sample implementation/results

Figures 1 and 4 are `sample_report_01` at issue widths 1 and 4. The sample runs commit, writeback, memory, execute, then issue. This simulator runs issue, execute, memory, writeback, then commit. The comments mark the cycle where each difference starts.

Issue width 1

Execute runs before memory, so a finished multiply is queued for writeback ahead of a load that finishes in the same cycle. The second `Ld F1` is `bus busy [16]`: the sample writes that load back on cycle 16, and we write the multiply back instead.

Issue runs before execute, so a reservation station freed during execute cannot be reused until the next cycle. The third `Add.d F3` is `no reservation station [15-16]` and issues on cycle 17. The sample issues it on cycle 16.

Issue width 4

Four results can write back together, so the multiply and the load no longer collide. What remains is the same stage order.

Writeback runs after execute, so a result cannot be used in the cycle it is written back. The second `Addi` starts on cycle 5. The sample starts it on cycle 4, the cycle the first `Addi` writes back.

A station freed during execute is issued on the next cycle. That wait is `no reservation station [4-10]`, `[11-16]`, and `[17]`.

Other discrepancies

The other timing differences follow two choices. They show up on the other programs and the other issue widths in the same way.

Who goes first is the stage order. Execute queues a finished result before memory does, so that result takes the bus and the other one is `bus busy`. A station or a unit freed during execute can be issued on the next cycle, so the instruction that wanted it carries `no reservation station` for the cycles it waited.

A reservation-station count and a functional-unit count are separate stops. A full station blocks issue. A busy unit blocks an instruction that already has a station, which is `no integer adder`, `no fp adder`, `no fp multiplier`, or `no load/store unit`. The floating-point units are pipelined, so each one still accepts one new operation per cycle. The integer adder and the load/store units are not. Where the sample lets a different instruction go first, or applies only one of those caps, the cycles shift for that reason.